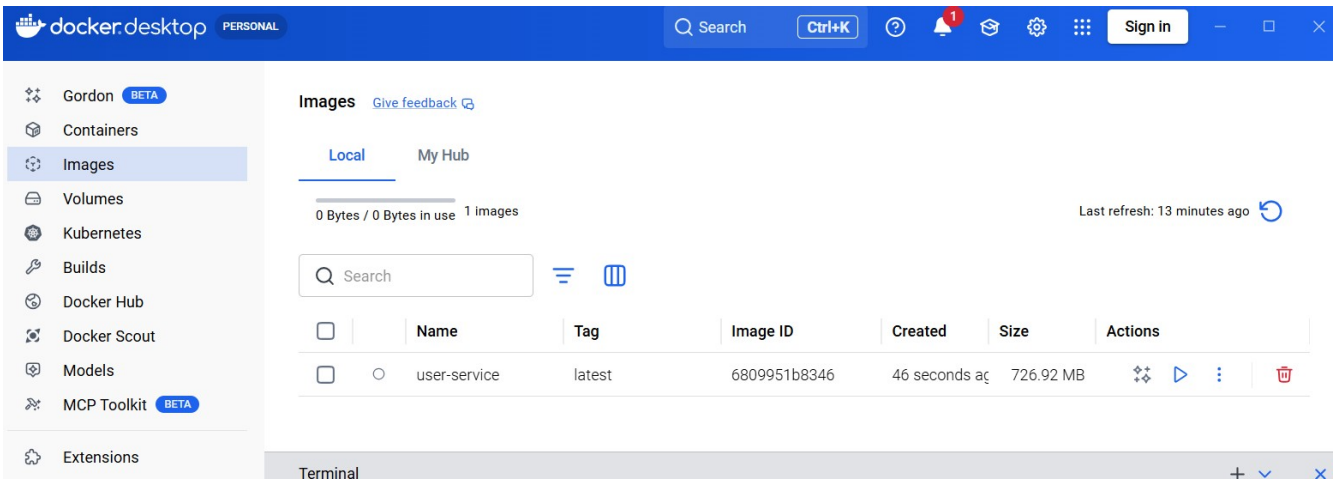
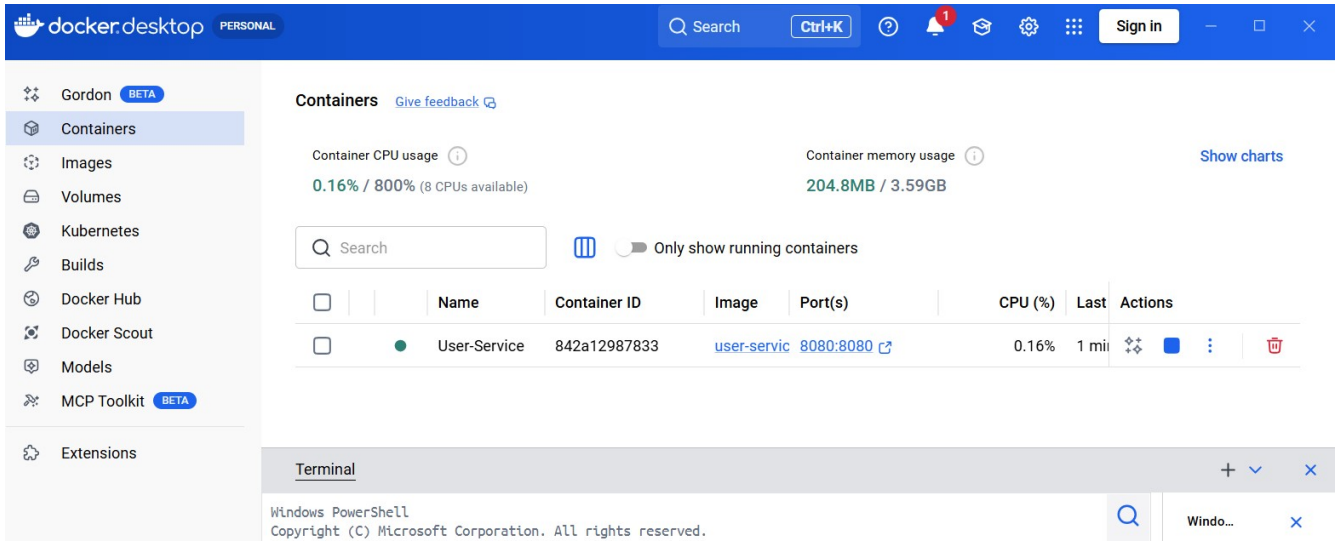
Docker Setup

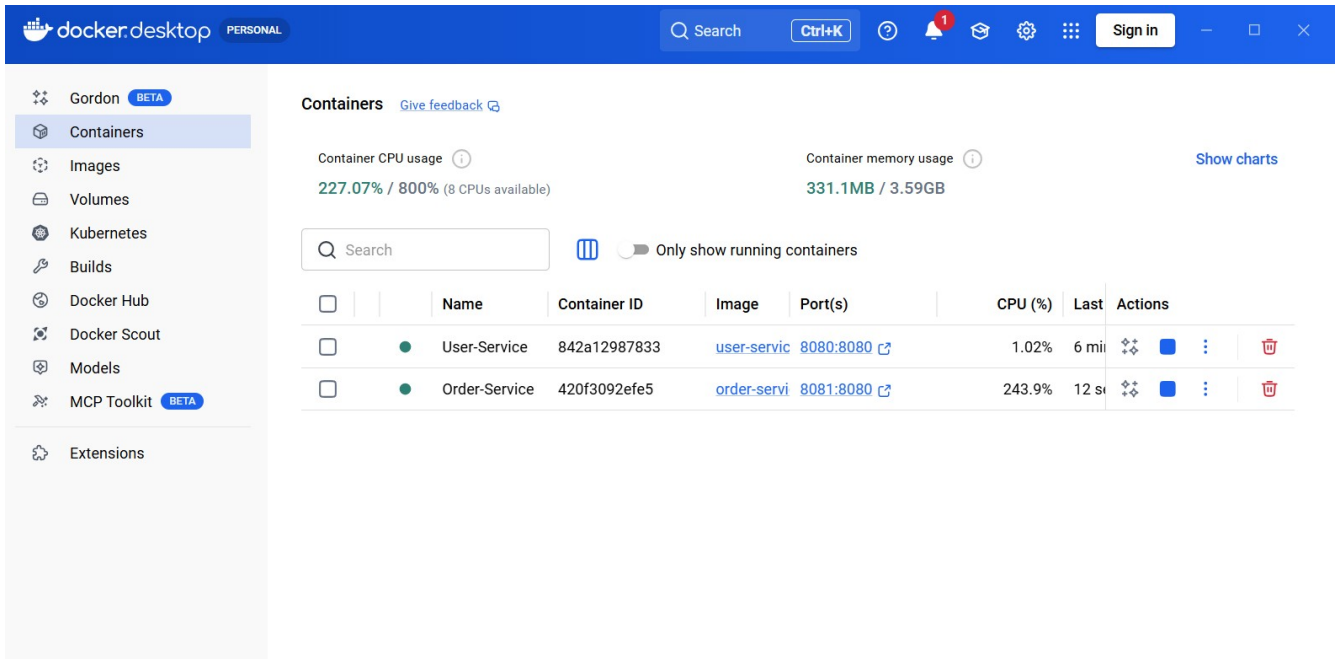
docker build -t user-service ./ecommerce

```
57

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    PROMPT FLOW

PS C:\Users\saito\source\repos\JavaMicroservices> docker build -t user-service ./ec
ommerce
 => => sha256:53405ff8853b1f58bfdc8d736024ea0d1b6df7fa5dedfb 2.28kB / 2.28kB  0.4s
 => => sha256:6acd3bd6814d41ef248fdd907c1157cbd4d90e1cc3e 59.77MB / 91.49MB  29.1s
 => => sha256:36b2ab561e1f5769d12480d85e27bf1a14a7757ff69b3acea0 129B / 129B  0.6s
 => => sha256:a150fdcc19ea38586861ab60f38c860b790aae07d0d 14.30MB / 14.30MB  14.4s
 => => sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0 3.86MB / 3.86MB  9.8s
 => => extracting sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa  1.8s
 => => extracting sha256:a150fdcc19ea38586861ab60f38c860b790aae07d0df506d623  3.0s
 => [internal] load build context                                             0.1s
 => => transferring context: 1.69kB                                           0.1s
```



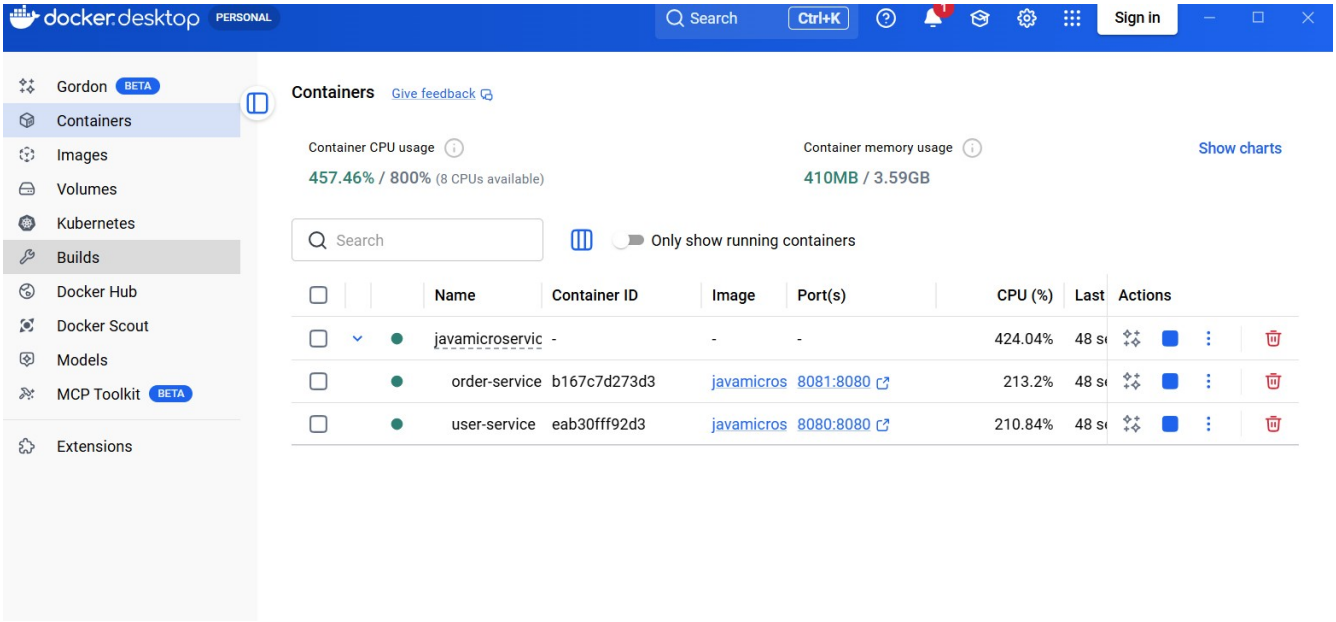
docker run -p 8080:8080 --name User-Service user-service






```
PS C:\Users\saito\source\repos\JavaMicroservices> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS
      PORTS                                       NAMES
420f3092efe5   order-service   "/__cacert_entrypoin…"   52 seconds ago   Up 51 seco
nds   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp   Order-Service
842a12987833   user-service    "/__cacert_entrypoin…"   6 minutes ago    Up 6 minut
es    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   User-Service
PS C:\Users\saito\source\repos\JavaMicroservices>
```

Docker-compose to start everything up in a single command
docker-compose up –build



To find logs of failing containers: docker-compose logs user-service

Now rebuild and restart user-service:

docker compose build user-service
docker compose up -d user-service
docker compose logs -f user-service

C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d "{\"name\":\"Sarah\"}"
{"id":1,"name":"Sarah","orders":null}

```
C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8080/us
ers -H "Content-Type: application/json" -d "{\"name\":\"Sarah\"}"
{"id":1,"name":"Sarah","orders":null}
C:\Users\saito\source\repos\JavaMicroservices>
```

Sarah has no order

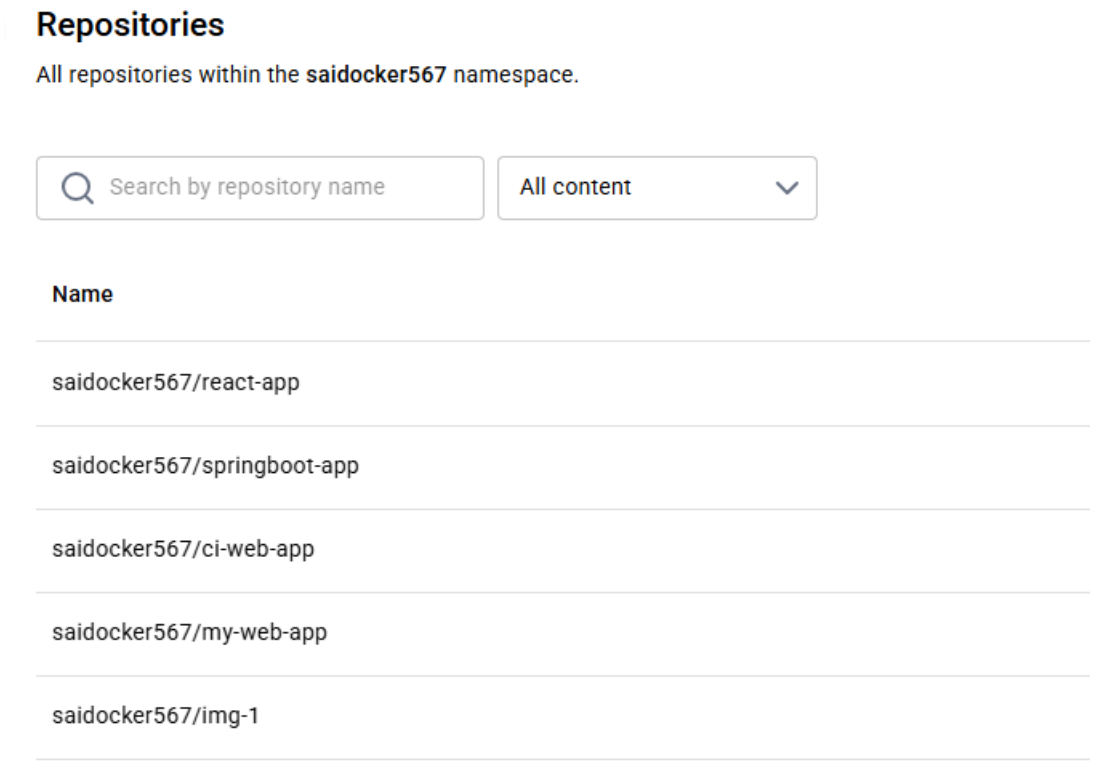
curl -X POST http://localhost:8081/orders -H "Content-Type: application/json" -d "{\"userID\":\"1\", \"item\":\"Apple\"}"

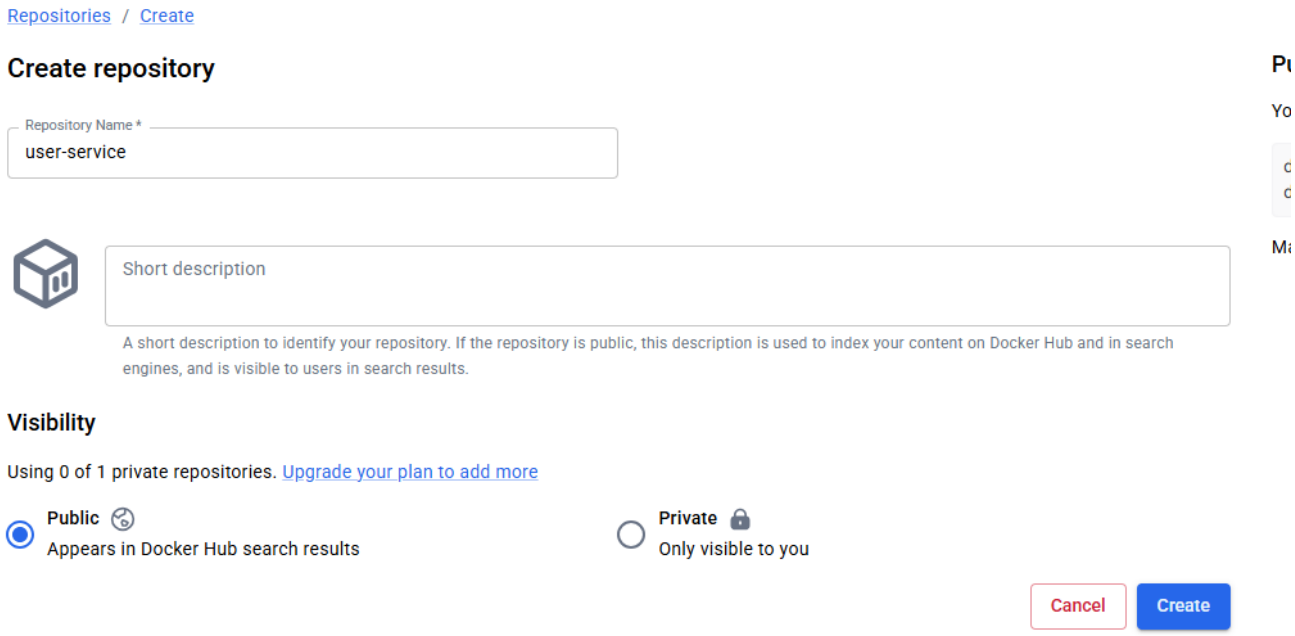
```
curl -X POST http://localhost:8081/orders -H "Content-Type: application/json" -d "{\"userID\":\"1\", \"item\":\"Apple\"}"
{"id":1,"item":"Apple","userID":1}

C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d "{\"name\":\"Sarah\"}"
{"id":1,"name":"Sarah","orders":null}
C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d "{\"name\":\"Amanda\"}"
{"id":2,"name":"Amanda","orders":null}
C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8081/orders -H "Content-Type: application/json" -d "{\"userID\":\"1\", \"item\":\"Orange\"}"
{"id":1,"item":"Orange","userID":1}
C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8081/orders -H "Content-Type: application/json" -d "{\"userID\":\"1\", \"item\":\"Phone\"}"
{"id":2,"item":"Phone","userID":1}
C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8081/orders -H "Content-Type: application/json" -d "{\"userID\":\"2\", \"item\":\"Orange\"}"
{"id":3,"item":"Orange","userID":2}

C:\Users\saito\source\repos\JavaMicroservices>curl -X GET http://localhost:8081/orders/user/1 -H "Content-Type: application/json"
[{"id":1,"item":"Orange","userID":1},{"id":2,"item":"Phone","userID":1}]
C:\Users\saito\source\repos\JavaMicroservices>curl -X GET http://localhost:8081/orders/user/1 -H "Content-Type: application/json"


C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d "{\"name\":\"Sarah\"}"
{"id":1,"name":"Sarah","orders":null}
C:\Users\saito\source\repos\JavaMicroservices>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d "{\"name\":\"Amanda\"}"
{"id":2,"name":"Amanda","orders":null}
```

<u>We are only passing-in UserID but we are pulling the Orders also</u>

```
C:\Users\saito\source\repos\JavaMicroservices>curl -X GET http://localhost:8080/users/1 -H "Content-Type: application/json"
{"id":1,"name":"Sarah","orders":[{"id":1,"item":"Orange"},{"id":2,"item":"Phone"}]}
C:\Users\saito\source\repos\JavaMicroservices>curl -X GET http://localhost:8080/users/2 -H "Content-Type: application/json"
{"id":2,"name":"Amanda","orders":[{"id":3,"item":"Orange"}]}
C:\Users\saito\source\repos\JavaMicroservices>curl -X GET http://localhost:8080/users/2 -H "Content-Type: application/json"
{"id":2,"name":"Amanda","orders":[{"id":3,"item":"Orange"},{"id":4,"item":"Apple"}]}
```

Pushing Docker images into Docker hub
Right now, it only works locally, but we want them to work outside also in public spaces



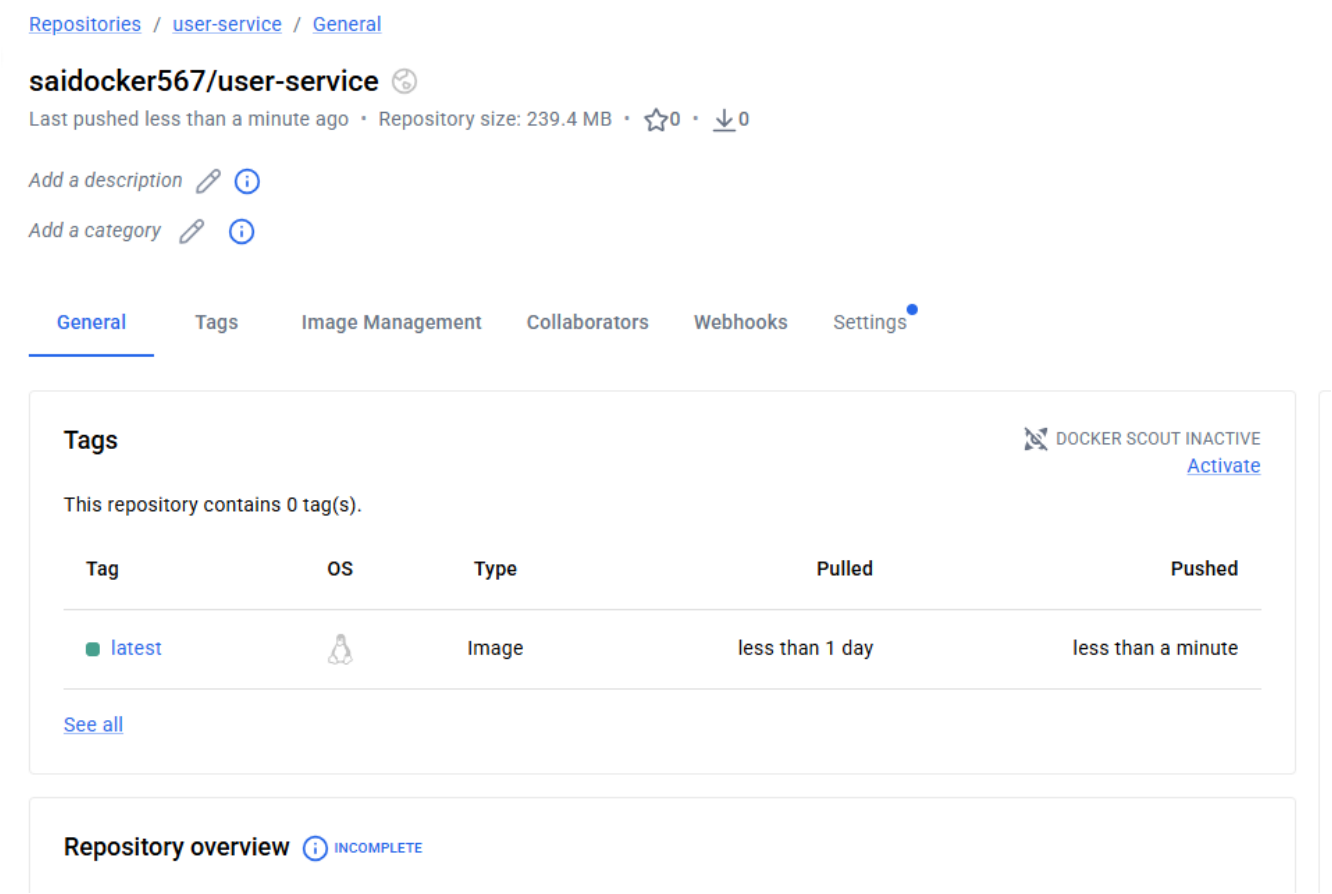


docker login
https://login.docker.com/activate

Enter code

docker images

```
                                        i Info →   U  In Use
IMAGE                          ID          DISK USAGE   CONTENT SIZE   EXTRA
javamicroservices-order-service:latest
                               e2162e5892fa      727MB         251MB    U
javamicroservices-user-service:latest
                               7b4c3e4bb5cf      727MB         251MB    U
PS C:\Users\saito\source\repos\JavaMicroservices> docker tag javamicroservices-order-service:latest
saidocker567/order-service
PS C:\Users\saito\source\repos\JavaMicroservices> docker tag javamicroservices-order-service:latest
saidocker567/order-service
PS C:\Users\saito\source\repos\JavaMicroservices> docker tag javamicroservices-user-service:latest
saidocker567/user-service
PS C:\Users\saito\source\repos\JavaMicroservices> docker push saidocker567/order-service
Using default tag: latest
```



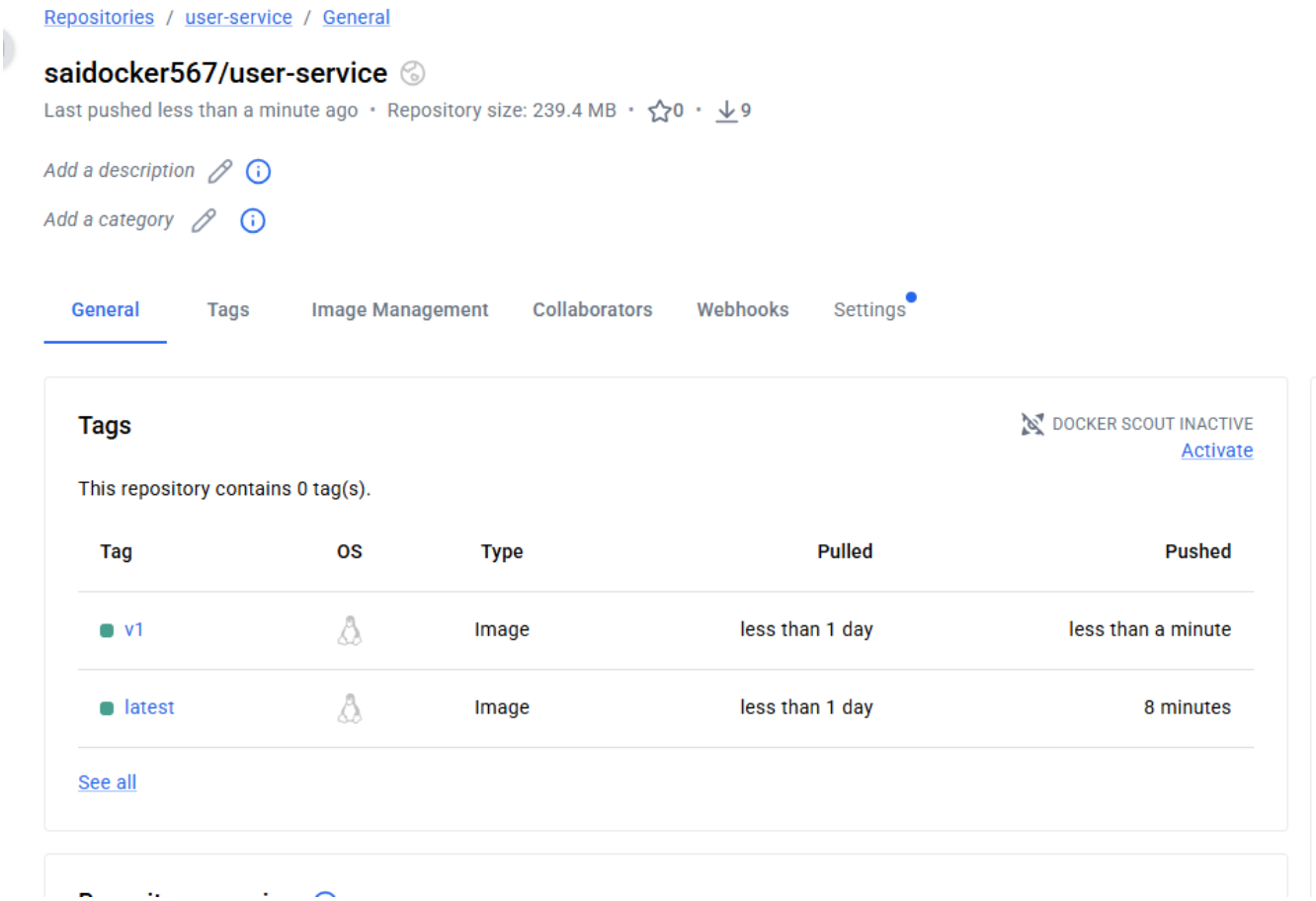
Next step is to automate pushing of images

Deploying using Docker compose

```
services:
  user-service:
    build: ./ecommerce
    image: saidocker567/user-service:latest
    ports:
      - "8080:8080"
    container_name: user-service
  order-service:
    build: ./orderservice
    image: saidocker567/order-service:latest
    ports:
      - "8081:8080"
    container_name: order-service
```

docker compose build
docker compose push

image : v1 is pushed as well



After pushing to Docker hub, i can easily integrate into CI pipeline